

提高级 CSP-S 第 17 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

- 在 Linux 系统终端中,用于删除或移动文件的命令语句是()。
A. cd B. ls C. rm D. rv
- 下列不属于视频文件格式后缀的是()。
A. mp3 B. wmv C. mpeg D. mp4
- $(101100101110)_2$ 与 $(A4E)_{16}$ 进行逻辑异或运算的结果是()。
A. $(000101101110)_2$ B. $(11301540)_8$
C. $(352)_{10}$ D. $(960)_{16}$
- 下列四个表达式对 int 变量 x 作用与其他三项不一样的是()。
A. $x \wedge = x \& 1$ B. $x = x / 2 * 2$ C. $x \& = \sim 1$ D. $x = x >> 1 << 1$
- 若一个二叉树的直径长度为 8,则这颗二叉树中至多有()个节点。
A. 30 B. 31 C. 46 D. 47
- 以下排序算法中,()不是原地排序算法,即额外空间复杂度为 $O(1)$ 的排序算法。
A. 快速排序 B. 堆排序 C. 希尔排序 D. 以上选项都不正确
- 由数字 0~9(同一数字不可以重复使用,允许前导零)所组成的四位数中有()个是 99 的倍数。
A. 72 B. 80 C. 84 D. 90
- 最朴素的 Bellman-Ford 算法和 SPFA 算法的最坏时间复杂度分别为(),其中 $|V|$ 表示图的点数, $|E|$ 表示图的边数。
A. $O(|V|+|E|), O(|V|+|E|)$ B. $O(|V|+|E|), O(|V|*|E|)$
C. $O(|V|*|E|), O(|V|+|E|)$ D. $O(|V|*|E|), O(|V|*|E|)$
- 1945 年,()首先提出了“存储程序”的概念和二进制原理,被誉为“电子计算机之父”。
A. 冯·诺伊曼 B. 克劳德·香农 C. 图灵 D. 欧拉
- 有如下递归代码,则 $F(51569)$ 的结果在以下哪个范围内()。
01 int F(int x){
02 if(x==1) return 1;
03 return (F(x/2)+F((x+1)/2))%233;
04 }

A. $[0, 59)$

B. $[59, 118)$

C. $[118, 177)$

D. $[177, 233)$

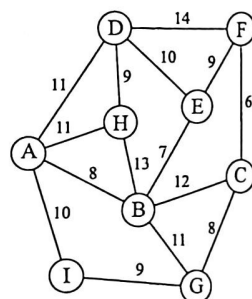
- 如右图所示,这张无向图的节点为 A, B, ..., I, 其中每条边的权值都标在图中,则这张图的最小生成树的边权和为()。

A. 63

B. 64

C. 65

D. 66



- 竞赛图是一种特殊的有向图,其中每两个点之间都会有恰好一条有向边,那么含有 5 个点的强连通竞赛图个数对 32 取模的结果为()。
A. 0 B. 8 C. 16 D. 24
- 依次抛出四个六面骰子,按照抛出顺序将骰子上的数值记为 a, b, c, d。则 $a < b, b > c, c < d$ 同时成立的概率为()。
A. $\frac{95}{648}$ B. $\frac{4}{27}$ C. $\frac{5}{27}$ D. $\frac{1}{6}$
- 下列说法正确的是()。
A. 用字符数组储存字符串时,最少只需要开字符串相同长度大小的字符数组
B. 迭代加深搜索与广度优先搜索是等价的,但是空间复杂度的需求大

C. 未进行路径压缩的并查集, 查询祖先的最坏复杂度是 $O(1)$ 的

D. 哈希表在遇到哈希冲突时, 可以用寻址法或挂链法解决

15. 下列情况不会被 CCF 禁赛的是()。

A. 在代码中使用暴戾语言

B. 在考场上偷窥别人代码

C. 携带 U 盘进入考场

D. 询问监考员厕所在哪里

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 单选题每题 3 分, 共计 40 分)

1.

```
01  #include<bits/stdc++.h>
02  using namespace std;
03  int dp[100001], n, a[100001], cost, w;
04  int main() {
05      cin>>n>>w;
06      for(int i=1; i<=n; i++) cin>>a[i];
07      for(int i=1; i<=w; i++){
08          cost=1e9;
09          for(int j=1; j<=n; j++){
10              if(i-a[j]>=0)
11                  cost=min(cost, dp[i-a[j]]);
12          }
13          dp[i]=cost+1;
14      }
15      cout<<dp[w];
16      return 0;
17 }
```

保证 $1 \leq n \leq 1000, 1 \leq a[i] \leq w \leq 10000$, 请回答下列问题:

●判断题

(1) 如果去掉第 10 行的 if 语句, 那么程序仍然能够正常运行。()

(2) 程序开始前, 如果 dp 数组中存在不为 0 的值, 那么程序输出不会改变。()

(3) 如果 $w \leq$ 所有 $a[i]$ 之和, 那么程序的输出一定小于等于 n。()

(4) 该程序可以用于求解完全背包问题。()

●单选题

(5) (4 分) 如果输入为:

6 789

1 5 10 20 50 100

则输出结果为()。

A. 14

B. 15

C. 16

D. 17

(6) 不认为 n 和 w 同阶, 则程序的时间复杂度为()。

A. $O(n+w)$

B. $O(n * w)$

C. $O(w+n \log w)$

D. $O(n+w \log n)$

2.

```
01  #include<iostream>
02  #include<algorithm>
03  using namespace std;
04  const int MOD=1001;
05  int n;
06  int a[1000001], las[1000001];
```

```

07 long long f[1000001];
08 int main() {
09     cin>>n;
10     for (int i=1; i<=n; i++) {
11         cin>>a[i];
12     }
13     for (int i=1; i<=n; i++) {
14         if (!las[a[i]]) {
15             f[i] = (f[i-1]*2+1)%MOD;
16         } else {
17             f[i] = (f[i-1]*2-f[las[a[i]]-1] + MOD)%MOD;
18         }
19         las[a[i]] = i;
20     }
21     cout<<f[n]<<endl;
22     return 0;
23 }

```

除特殊声明外,假设输入的所有数都是不超过 10^6 的正整数,完成下面的判断题和单选题:

●判断题

- (1) 程序的输出一定是正整数。()
- (2) 若将数组 f 改成 int 类型的数组,那么程序的输出可能会改变。()
- (3) 当 $n \leq 9$ 的时候,程序的输出一定是奇数。()
- (4) 若把 a 倒序输入,那么程序的输出可能会改变。()

●单选题

- (5) 如果输入为:5 1 2 3 1 2,则输出结果为()。
 A. 26 B. 27 C. 28 D. 29
- (6) (4分)若 $n=181766$, $a[i]=i$,则程序的输出为()。
 A. 197 B. 493 C. 691 D. 822

3.

```

001 #include<bits/stdc++.h>
002 using namespace std;
003
004 const int N=100001;
005
006 int n, m, siz[N], vis[N];
007
008 int cnt1, head1[N], nxt1[N], ver1[N];
009 int cnt2, head2[N], nxt2[N], ver2[N];
010
011 int scc[N], scct, dfn[N], tot, low[N], stk[N], top;
012
013 int in[N];
014 long long sum[N];
015
016 void addedge1 (int u, int v) {

```

```
017     ver1[++cnt1]=v;
018     nxt1[cnt1]=head1[u];
019     head1[u]=cnt1;
020 }
021
022 void addedge2 (int u, int v) {
023     ver2[++cnt2]=v;
024     nxt2[cnt2]=head2[u];
025     head2[u]=cnt2;
026 }
027
028 void tarjan (int u) {
029     dfn[u]=low[u]=++tot;
030     stk[++top]=u;
031     for (int i=head1[u]; i; i=nxt1[i]) {
032         int v=ver1[i];
033         if (!dfn[v]) {
034             tarjan (v);
035             low[u]=min (low[u], low[v]);
036         } else if (!scc[v]) {
037             low[u]=min (low[u], dfn[v]);
038         }
039     }
040     if (dfn[u]==low[u]) {
041         scct++;
042         do {
043             scc[stk[top]]=scct;
044         } while (stk[top--]!=u);
045     }
046 }
047
048 void topo() {
049     queue<int>q;
050     for (int i=1; i<=scct; i++) {
051         if (!in[i]) {
052             q.push (i);
053         }
054     }
055     while (!q.empty()) {
056         int u=q.front();
057         q.pop();
058         sum[u]++;
059         for (int i=head2[u]; i; i=nxt2[i]) {
060             int v=ver2[i];
061             in[v]--;
```

```

062         sum[v]+=sum[u];
063         if (!in[v]) {
064             q.push (v);
065         }
066     }
067 }
068 }
069
070 int main() {
071     cin>>n>>m;
072
073     for (int i=1, u, v; i<=m; i++) {
074         cin>>u>>v;
075         addedge1 (u, v);
076     }
077
078     for (int i=1; i<=n; i++) {
079         if (!dfn[i]) {
080             tarjan (i);
081         }
082     }
083
084     for (int i=1; i<=n; i++) {
085         siz[scc[i]]++;
086     }
087
088     long long ans=1ll*n*n;
089     for (int i=1; i<=scct; i++) {
090         ans -=1ll*siz[i]*siz[i];
091     }
092
093     cout<<ans<<endl;
094
095     for (int u=1; u<=n; u++) {
096         for (int i=head1[u]; i; i=nxt1[i]) {
097             if (int v=ver1[i]; scc[u]!=scc[v]) {
098                 addedge2 (scc[v], scc[u]);
099                 in[scc[u]]++;
100             }
101         }
102     }
103
104     topo();
105     for (int i=1; i<=n; i++) {
106         cout<<sum[scc[i]]<<"\n "[i<n];

```

```

107     }
108
109     return 0;
110 }

```

提示:本题的输入为一张有向图,且点数和边数都不超过 5000,保证无重边和自环,完成下面的判断题和单选题:

● 判断题

- (1) 程序的输出的第一行一定是偶数。()
- (2) 程序的输出的第二行一定是若干非负整数。()
- (3) (2.5 分)如果输入的图是一张弱连通图(即将每条边看成无向边时整张图联通),那么程序仅会进入第 80 行一次。()
- (4) 如果将程序的第 37 行替换成第 35 行,那么程序的输出一定不会改变。()

● 单选题

- (5) 若认为 n, m 同阶,则该算法的最坏时间复杂度为()。

A. $O(n)$	B. $O(n \log n)$
C. $O(n^2)$	D. 以上选项均不正确
- (6) (4 分)程序的输入为:

```

5 7
5 4
1 2
2 3
1 3
3 4
4 5
3 5

```

则程序输出所有数的和为()。

- | | | | |
|-------|-------|-------|-------|
| A. 34 | B. 35 | C. 36 | D. 37 |
|-------|-------|-------|-------|

三、完善程序(单选题,每小题 3 分,共计 30 分)

1. 开玩笑

你用过 QQ 吗? 在 QQ 群里,管理员可以禁言用户。在 Boboniu 的 QQ 群里,小 D 每天都开 Boboniu 的玩笑。

小 D 会在群里待 n 天, Boboniu 的心情是 k 。在第 i 天,如果小 D 没被禁言,他会开一个严重程度为 $a[i]$ 的玩笑。

如果开的玩笑严重程度大于 k ,他就会被 Boboniu 禁言 m 天,也就是说,在第 $i+1, i+2, \dots, \min(i+m, n)$ 天,他都会被禁言。

你可以将序列 a 重排,求开的所有玩笑的严重程度之和的最大值。

注意,被禁言当天,玩笑的严重程度也应当计算进答案中。

数据范围: $1 \leq m \leq n \leq 10^5, 0 \leq k, a_i \leq 10^9$ 。

提示:使用贪心、枚举算法解决该问题。

```

01 #include<bits/stdc++.h>
02 using namespace std;
03
04 const int MAXN=1e5+5;
05 int big[MAXN], small[MAXN];
06 long long sum[MAXN];
07 int p1=1, p2=1;

```

```

08  int n, m, k, x;
09
10  int main() {
11      cin>>n>>m>>k;
12      for(int i=1; i<=n; i++) {
13          cin>>x;
14          if(x<=k) {
15              ①
16          } else {
17              big[p2++]=x;
18          }
19      }
20      sort(small+1, small+1+p1, greater<int>());
21      sort(big+1, big+1+p2, greater<int>());
22      for(int i=1; i<=p1; i++) {
23          ②
24      }
25      long long ans=sum[p1], cur=0;
26      for(int i=1; i<=p2; i++) {
27          cur+= ③;
28          int days= ④ +1;
29          if(days>n) {
30              break;
31          }
32          int left=min(n-days, p1);
33          ans=max(ans, ⑤);
34      }
35      cout<<ans<<endl;
36      return 0;
37  }

```

(1) ① 处应填()。

- A. big[++p1] = x;
C. small[++p1] = x;

- B. big[p1++] = x;
D. small[p1++] = x;

(2) ② 处应填()。

- A. sum[i] = sum[i-1] + small[i]
B. sum[i+1] = sum[i] + small[i+1]
C. sum[i] = sum[i-1] + big[i]
D. sum[i+1] = sum[i] + big[i]

(3) ③ 处应填()。

- A. small[i] B. big[i]

- C. small[i+1] D. big[i+1]

(4) ④ 处应填()。

- A. (i-1) * m B. i * (m+1)

- C. i * m D. (i-1) * (m+1)

(5) ⑤ 处应填()。

- A. sum[left+1] + cur
C. sum[left-1] + cur

- B. sum[left] + cur
D. sum[left] + cur - 1

2. 方案统计

给出一个十进制的 n 位数,要求将该 n 位数拆分成若干区间,每个区间表示一个数字,且需满足:

- (1) 数字要严格递增;
- (2) 数字都是正整数;
- (3) 无前导零。

请你计算所有可能的方案数量,答案对 10^9+7 取模。

数据范围: $1 \leq n \leq 5000$ 。

提示:从后往前 dp,采用前缀和优化 dp 的思路,使用记忆化搜索实现。

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 const int MOD=1e9+7;
04 char str[5039];
05 long long f[5039][5039], g[5039][5039];
06 int lcp[5039][5039];
07 int n, a[5039];
08 bool compare (int x, int y) {
09     if (lcp[x][y]>= ① ) {
10         return false;
11     }
12     if (a[x+lcp[x][y]]<a[y+lcp[x][y]]) {
13         return true;
14     }
15     if (a[x+lcp[x][y]]>a[y+lcp[x][y]]) {
16         return false;
17     }
18     return false;
19 }
20 long long sum (int, int);
21 long long dp (int, int);
22 long long sum (int i, int j) {
23     if (i>j) {
24         return 0;
25     }
26     if (i<1) {
27         return sum (1, j);
28     }
29     if (g[i][j]!=-1) {
30         return g[i][j];
31     }
32     return g[i][j]=(sum (i+1, j)+dp (i, j))%MOD;
33 }
34 long long dp (int i, int j) {
35     if ( ② ) {
36         return 0;
37     }

```



```

38     if ( ③ ) {
39         return 1;
40     }
41     if (f[i][j]!=-1) {
42         return f[i][j];
43     }
44     int k= ④;
45     long long ans=sum (k+1, i-1);
46     if (k>0) {
47         if (compare (k , i) ) {
48             ans=(ans+dp (k, i-1) )% MOD;
49         }
50     }
51     return f[i][j]=ans;
52 }
53 int main() {
54     scanf("%d%s", &n, str+1);
55     for (int i=1; i<=n; i++) {
56         a[i]=str[i]-'0';
57     }
58     for (int i=n; i>=1; i--) {
59         for (int j=n; j>=1; j--) {
60             if ( ⑤ ) {
61                 lcp[i][j]=lcp[i+1][j+1]+1;
62             } else {
63                 lcp[i][j]=0;
64             }
65         }
66     }
67     memset (f, -1, sizeof f);
68     memset (g, -1, sizeof g);
69     cout<<sum (1, n)<<endl;
70     return 0;
71 }

```

(1) ① 处应填()。

A. n-x+1 B. n-y+1 C. y-x D. y-x+1

(2) ② 处应填()。

A. i==0 B. a[i]==0 C. i>j D. a[i]>a[j]

(3) ③ 处应填()。

A. i==0 B. i==1 C. j==0 D. i==j

(4) ④ 处应填()。

A. 2*i-j-1 B. 2*i-j C. i-j D. lcp[i][j]

(5) ⑤ 处应填()。

A. a[n-i+1]!=a[n-j+1] B. a[n-i+1]==a[n-j+1]
C. a[i]!=a[j] D. a[i]==a[j]